

Circuits with Qiskit

Move from diagrams to code. Build, transpile, and simulate circuits in Qiskit, then read the transpiler's output like a compiler log.

EXAMPLE NOTES

Written by the QuantaVerse team to show what a module's notes can look like: each lesson's theory in brief, every formula typeset and numbered, and the key ones gathered on a formula sheet at the end. Professors can upload their own notes for this module on its page, and students will see them alongside these.

5.1	QuantumCircuit	2
	The object everything else in Qiskit hangs off.	
5.2	Adding gates	3
	The method names, and the argument order that trips everyone once.	
5.3	Running it	4
	Shots, samplers, and getting counts back out.	
5.4	The statevector simulator	5
	Exact amplitudes, no sampling, and why it is the one you learn with.	
5.5	Basis gates	6
	The short list a real machine can actually perform.	
5.6	Transpilation	7
	The compiler step between what you wrote and what runs.	
5.7	Depth and width	8
	The two numbers that decide whether a circuit survives the hardware.	
5.8	Reading a circuit back	9
	Drawing, inspecting, and the fact that a circuit is data.	
Σ	Formula sheet	10
	Every key formula of the module, gathered at the end.	

CONVENTIONS

Code is Qiskit 1.x and 2.x. Qiskit numbers qubits from 0 and orders basis states little-endian, $|q_{n-1} \cdots q_0\rangle$, in amplitudes, matrices and count strings alike. d is circuit depth, n the number of qubits and N the number of shots.

LESSON 5.1

QuantumCircuit

KEY IDEA

A *QuantumCircuit* is a list of instructions over numbered qubits and classical bits. It describes a computation; nothing runs until it is handed to a simulator or a device.

CREATING A CIRCUIT

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(2, 2)  # 2 qubits, 2 classical bits
print(qc.num_qubits, qc.num_clbits)  # 2 2
```

Every qubit starts in $|0\rangle$, so a fresh circuit with n qubits represents the all-zero state, a vector of 2^n amplitudes with a single 1 in it.

$$|\psi_0\rangle = |0\rangle^{\otimes n} = |0 \cdots 0\rangle \quad (5.1.1)$$

the initial state of every circuit

Registers can be named and combined, which matters once a circuit has an input register, a work register and ancillas.

NAMED REGISTERS

```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit

data = QuantumRegister(3, "data")
anc = QuantumRegister(1, "anc")
out = ClassicalRegister(3, "out")
qc = QuantumCircuit(data, anc, out)  # qubits data[0..2], anc[0]
```

DEFINITION · INSTRUCTION

One gate, measurement, reset or barrier, together with the qubits and bits it acts on. A circuit is an ordered list of them, available as `qc.data` (lesson 5.8).

PROPERTY	MEANING	CALL
width	qubits plus classical bits	<code>qc.width()</code>
size	number of operations	<code>qc.size()</code>
depth	number of layers (lesson 5.7)	<code>qc.depth()</code>
operation counts	how many of each gate	<code>qc.count_ops()</code>

$$U_{\text{circuit}} = U_k \cdots U_2 U_1 \quad \text{for instructions } 1, \dots, k \text{ in order} \quad (5.1.2)$$

what a measurement-free circuit represents

WORKED EXAMPLE · SIZING A REGISTER

A 20-qubit circuit represents a state of $2^{20} = 1\,048\,576$ amplitudes. At 16 bytes per complex amplitude that is 16 MB to simulate exactly, and every extra qubit doubles it.

LESSON 5.2

Adding gates

KEY IDEA

Gates are methods named after them, qubit arguments come after any angles, and for controlled gates the control comes first.

THE COMMON GATES

```
from math import pi

qc = QuantumCircuit(2, 2)
qc.h(0)          # Hadamard on qubit 0
qc.x(1); qc.y(1); qc.z(1)
qc.s(0); qc.sdg(0) # S and S-dagger
qc.t(0); qc.tdg(0) # T and T-dagger
qc.rz(pi / 4, 0)  # angle first, then the qubit
qc.cx(0, 1)       # control 0, target 1
qc.measure([0, 1], [0, 1]) # qubits, then classical bits
```

GATE	METHOD	ARGUMENTS
H, X, Y, Z, S, T	<code>h, x, y, z, s, t</code>	qubit
$S^\dagger, T^\dagger$	<code>sdg, tdg</code>	qubit
R_x, R_y, R_z	<code>rx, ry, rz</code>	angle, qubit
CNOT	<code>cx</code>	control, target
controlled-Z	<code>cz</code>	qubit, qubit (symmetric)
measurement	<code>measure</code>	qubit(s), classical bit(s)

Because Qiskit is little-endian, the matrix of a two-qubit gate depends on which qubit is which. A CNOT from qubit 0 to qubit 1 is not the textbook CNOT matrix, which assumes the control is written first.

$$CX_{0 \rightarrow 1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \text{ in the basis } |q_1 q_0\rangle = |00\rangle, |01\rangle, |10\rangle, |11\rangle \quad (5.2.1)$$

control q_0 , the low bit

Angles can also be symbolic, bound to values later, so one circuit serves a whole parameter sweep.

PARAMETERS

```
from qiskit.circuit import Parameter

theta = Parameter("θ")
qc.ry(theta, 0)
bound = qc.assign_parameters({theta: 0.6435}) # about 90/10 (lesson 3.5)
```

WORKED EXAMPLE · READING AN INDEX

After `qc.x(0)` on two qubits, the state is $|q_1 q_0\rangle = |01\rangle$, index $x = 1$, not $|10\rangle$. After `qc.x(1)` instead it is $|10\rangle$, index 2. Reading the rightmost digit as qubit 0 avoids nearly every ordering bug.

LESSON 5.3

Running it

KEY IDEA

A sampler runs a measured circuit a number of times and returns counts: how often each bit string came out.

SAMPLING A BELL PAIR

```
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler

qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

sampler = StatevectorSampler()
result = sampler.run([qc], shots=1024).result()
counts = result[0].data.c.get_counts() # 'c' is the register's name
print(counts) # e.g. {'00': 519, '11': 505}
```

Counts are samples, not probabilities. Dividing by the number of shots estimates each probability, with the statistical error of lesson 2.3.

$$\hat{p}_x = \frac{n_x}{N}, \quad \text{SE}(\hat{p}_x) = \sqrt{\frac{\hat{p}_x(1 - \hat{p}_x)}{N}} \quad (5.3.1)$$

estimating a probability from counts

Many quantities of interest are expectation values rather than single probabilities. For observables that are products of Z , each outcome contributes $+1$ or -1 according to the parity of the measured bits.

$$\langle Z_0 Z_1 \rangle = p_{00} - p_{01} - p_{10} + p_{11} \quad (5.3.2)$$

a parity expectation from counts

$$\langle Z_0 \rangle = p_{00} + p_{10} - p_{01} - p_{11} \quad (\text{bit } q_0 \text{ is the rightmost}) \quad (5.3.3)$$

a single-qubit expectation, marginalised

DEFINITION · PRIMITIVES

Qiskit's two primitives are the Sampler, which returns counts or quasi-probabilities, and the Estimator, which returns expectation values of observables directly. On hardware, the same interface runs on real devices with error mitigation.

WORKED EXAMPLE · READING THE BELL COUNTS

From $\{ '00': 519, '11': 505 \}$: $\hat{p}_{00} = 519/1024 \approx 0.507$ with standard error ≈ 0.016 , consistent with $\frac{1}{2}$.
And $\langle Z_0 Z_1 \rangle = (519 + 505)/1024 = 1$: the two bits always agree.

LESSON 5.4

The statevector simulator

KEY IDEA

A statevector simulator multiplies the circuit's gates into the vector of 2^n amplitudes and returns them exactly: no shots and no noise, only memory.

$$|\psi\rangle = U_{\text{circuit}}|0\rangle^{\otimes n}, \quad p(x) = |\langle x|\psi\rangle|^2 \quad (5.4.1)$$

what the simulator computes

EXACT AMPLITUDES

```
from qiskit.quantum_info import Statevector

bell = qc.remove_final_measurements(inplace=False)
sv = Statevector.from_instruction(bell)
print(sv) # [0.707+0j, 0, 0, 0.707+0j]
print(sv.proBABILITIES_DICT()) # {'00': 0.5, '11': 0.5}
```

Sampled and exact results answer different questions. The sampler shows what an experiment would record; the statevector shows the state itself, including phases that no single measurement reveals.

	SAMPLER, 1024 SHOTS	STATEVECTOR
Output	{'00': 511, '11': 513}	$0.707 00\rangle + 0.707 11\rangle$
Phases visible	no	yes
Error	about ± 0.016 per probability	none, up to rounding
Cost	grows with shots	grows as 2^n

Each gate touches every amplitude, so applying one costs time proportional to 2^n . Memory is the hard wall.

$$\text{memory} = 16 \cdot 2^n \text{ bytes} \quad (5.4.2)$$

complex128 amplitudes: 16 GB at $n = 30$, 16 TB at $n = 40$

$$\text{time per gate} = O(2^n), \quad \text{circuit of } G \text{ gates} = O(G \cdot 2^n) \quad (5.4.3)$$

the cost of exact simulation

Other simulators trade exactness or generality for size: stabiliser simulators handle Clifford circuits of thousands of qubits (lesson 3.7), and tensor-network simulators handle circuits with little entanglement.

WORKED EXAMPLE · SEEING A PHASE

$|\Phi^+\rangle$ and $|\Phi^-\rangle$ give identical counts, 00 and 11 about half each. Their statevectors differ, $[0.707, 0, 0, 0.707]$ against $[0.707, 0, 0, -0.707]$, and that sign is exactly what superdense coding uses to carry a second bit.

LESSON 5.5

Basis gates

KEY IDEA

A device implements a handful of native gates. Every other gate must be rewritten in terms of them before the circuit can run.

A typical superconducting basis is $\{R_z, \sqrt{X}, X\}$ plus one two-qubit gate, CX on some devices and CZ or ECR on others. R_z is usually applied in software, as a change of reference frame, so it takes no time and adds no error.

$$\sqrt{X} = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}, \quad (\sqrt{X})^2 = X \quad (5.5.1)$$

the SX gate

$$H = e^{i\pi/4} R_z\left(\frac{\pi}{2}\right) \sqrt{X} R_z\left(\frac{\pi}{2}\right) \quad (5.5.2)$$

the Hadamard, in basis gates

$$U = e^{i\alpha} R_z(\lambda_1) \sqrt{X} R_z(\lambda_2) \sqrt{X} R_z(\lambda_3) \quad (5.5.3)$$

any single-qubit gate: at most two $\sqrt{X}$ pulses

Two-qubit gates convert into each other with single-qubit gates on the target.

$$CX = (I \otimes H) CZ (I \otimes H) \quad (5.5.4)$$

CNOT from controlled-Z: Hadamards on the target

REWRITING INTO A BASIS

```
from qiskit import transpile

native = transpile(qc, basis_gates=["rz", "sx", "x", "cx"], optimization_level=1)
print(native.count_ops())
```

WORKED EXAMPLE · CHECKING THE HADAMARD DECOMPOSITION

With $\sqrt{X} = \frac{1}{\sqrt{2}} \begin{pmatrix} e^{i\pi/4} & e^{-i\pi/4} \\ e^{-i\pi/4} & e^{i\pi/4} \end{pmatrix}$ and $R_z(\frac{\pi}{2}) = \text{diag}(e^{-i\pi/4}, e^{i\pi/4})$, each entry of $R_z \sqrt{X} R_z$ picks up one phase from each side.

The result is $\frac{e^{-i\pi/4}}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = e^{-i\pi/4} H$, so multiplying by $e^{i\pi/4}$ gives H exactly.

A gate that looks simple on paper can be expensive natively, and the reverse. The count that matters is the count after rewriting, which is why lesson 5.6 reads the transpiler's output.

LESSON 5.6

Transpilation

KEY IDEA

The transpiler maps a circuit onto a device: it chooses physical qubits, inserts SWAPs where the wiring requires them, rewrites everything into basis gates and simplifies the result.

STAGE	WHAT IT DOES
init	unroll to one- and two-qubit gates
layout	assign each virtual qubit to a physical qubit
routing	insert SWAPs so every two-qubit gate acts on connected qubits
translation	rewrite into the basis gates
optimisation	cancel and merge gates
scheduling	place gates in time, if requested

Hardware qubits only interact with their neighbours on a *coupling map*. A two-qubit gate between distant qubits needs SWAPs, and each SWAP costs three CNOTs.

$$\text{SWAP} = CX_{0 \rightarrow 1} CX_{1 \rightarrow 0} CX_{0 \rightarrow 1} \quad (5.6.1)$$

a SWAP from three CNOTs

TRANSPILING FOR A LINE OF THREE QUBITS

```
from qiskit import transpile

qc = QuantumCircuit(3)
qc.h(0)
qc.cx(0, 2)  # qubits 0 and 2 are not connected below

line = [[0, 1], [1, 0], [1, 2], [2, 1]]
out = transpile(qc, coupling_map=line, basis_gates=["rz", "sx", "x", "cx"],
               optimization_level=3,  # 0 translates only, 3 searches hardest
               seed_transpiler=7)    # fixed, so the same run comes back
```

The output does the same thing as the input, up to a global phase and a relabelling of qubits: routing may also leave the qubits somewhere else at the end.

$$V = e^{i\gamma} P_{\text{final}} U P_{\text{initial}}^\dagger \quad (5.6.2)$$

transpiled circuit V against original U : permutations P from layout and routing

Every inserted gate adds error, so the transpiler's first goal is fewer two-qubit gates. To first order, a circuit runs cleanly with the product of its gates' success rates.

$$F \approx \prod_g (1 - \varepsilon_g) \approx \exp\left(-\sum_g \varepsilon_g\right) \quad (5.6.3)$$

estimated success probability from per-gate error rates ε_g

WORKED EXAMPLE · WHAT ROUTING COSTS

With two-qubit error $\varepsilon = 10^{-2}$, one SWAP adds three CNOTs: a factor of $(0.99)^3 \approx 0.970$. Ten SWAPs cost $(0.99)^{30} \approx 0.74$, a quarter of all runs lost to routing alone.

LESSON 5.7

Depth and width

KEY IDEA

Width is how many qubits a circuit needs. Depth is how many layers of gates it takes, and depth sets how long the qubits must stay coherent.

DEFINITION · DEPTH

Push every gate as early in time as it can go without passing a gate on the same qubit. The number of resulting layers is the depth. Gates on different qubits in the same layer run at the same time.

$$t_{\text{circuit}} \approx d \cdot t_{\text{layer}}, \quad P_{\text{coherent}} \approx e^{-t_{\text{circuit}}/T_2} \quad (5.7.1)$$

depth becomes time, and time becomes decoherence

DEPTH d	TIME AT $0.1 \mu\text{S}$ PER LAYER	e^{-t/T_2} WITH $T_2 = 100 \mu\text{S}$
10	$1 \mu\text{s}$	0.990
100	$10 \mu\text{s}$	0.905
1000	$100 \mu\text{s}$	0.368

The timing figures are illustrative, not those of a particular device.

Width and depth trade against each other. Extra ancilla qubits can let gates run in parallel. Preparing an n -qubit GHZ state is the standard example.

$$|\text{GHZ}_n\rangle = \frac{|0\rangle^{\otimes n} + |1\rangle^{\otimes n}}{\sqrt{2}} \quad (5.7.2)$$

the GHZ state

$$\text{chain of CNOTs: depth } n \quad \text{fan-out doubling the entangled set each layer: depth } 1 + \lceil \log_2 n \rceil \quad (5.7.3)$$

the same state, two depths (all-to-all connectivity)

A single figure of merit combines the two. A device has *quantum volume* 2^m when it reliably runs random circuits m qubits wide and m layers deep, reliably meaning the “heavy” outputs appear more than two-thirds of the time.

$$\text{QV} = 2^m, \quad \text{largest } m \text{ with } \Pr[\text{heavy output}] > \frac{2}{3} \quad (5.7.4)$$

quantum volume

WORKED EXAMPLE · DEPTH BY HAND

On three qubits: H on q0; CX(0,1); H on q2; CX(1,2).

Layer 1: H on q0 and H on q2 together. Layer 2: CX(0,1). Layer 3: CX(1,2), which waits for q1. The depth is 3, though there are four gates.

LESSON 5.8

Reading a circuit back

KEY IDEA

A circuit is a data structure: it can be drawn, iterated, compared, serialised and edited like any other.

INSPECTING A CIRCUIT

```
print(qc.draw("text"))
print(qc.count_ops())          # e.g. {'h': 1, 'cx': 1, 'measure': 2}

for instruction in qc.data:
    name = instruction.operation.name
    wires = [qc.find_bit(q).index for q in instruction.qubits]
    print(name, wires)         # h [0], then cx [0, 1], ...
```

Two circuits can look nothing alike and still perform the same operation. Comparing their unitaries settles it, and equivalence is only meaningful up to a global phase.

$$U \simeq V \Leftrightarrow \frac{|\text{Tr}(U^\dagger V)|}{2^n} = 1 \quad (5.8.1)$$

equal up to a global phase

$$F_{\text{pro}}(U, V) = \frac{|\text{Tr}(U^\dagger V)|^2}{d^2}, \quad d = 2^n \quad (5.8.2)$$

process fidelity: 1 for the same operation; the algorithm labs are marked with it

CHECKING EQUIVALENCE

```
from math import pi
from qiskit.quantum_info import Operator

a = QuantumCircuit(1); a.h(0)
b = QuantumCircuit(1); b.rz(pi / 2, 0); b.sx(0); b.rz(pi / 2, 0)
print(Operator(a).equiv(Operator(b))) # True: equal up to global phase
```

Circuits also serialise to OpenQASM, the text format most quantum toolchains read, which is how a circuit moves between Qiskit, other frameworks and hardware providers.

OPENQASM 3

```
from qiskit import qasm3
print(qasm3.dumps(qc))
# OPENQASM 3.0; include "stdgates.inc"; qubit[2] q; h q[0]; cx q[0], q[1]; ...
```

WORKED EXAMPLE · WHY THE GRADER USES FIDELITY

A learner's circuit V for a target $U = X$ might be $V = HZH$, which equals X exactly:

$|\text{Tr}(X^\dagger HZH)| = |\text{Tr}(I)| = 2$, so $F = 4/4 = 1$. A circuit $V = -iY$ gives $|\text{Tr}(X^\dagger (-iY))| = |-i \text{Tr}(XY)| = 0$, so $F = 0$, correctly scored as a different operation.

MODULE 5 · FORMULA SHEET

Formula sheet

The key results of the module, numbered as they appear in the lessons.

the initial state of every circuit $|\psi_0\rangle = |0\rangle^{\otimes n} = |0 \dots 0\rangle$ (5.1.1)

control q_0 , the low bit $CX_{0 \rightarrow 1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$ in the basis $|q_1 q_0\rangle = |00\rangle, |01\rangle, |10\rangle, |11\rangle$ (5.2.1)

estimating a probability from counts $\hat{p}_x = \frac{n_x}{N}, \quad \text{SE}(\hat{p}_x) = \sqrt{\frac{\hat{p}_x(1 - \hat{p}_x)}{N}}$ (5.3.1)

a parity expectation from counts $\langle Z_0 Z_1 \rangle = p_{00} - p_{01} - p_{10} + p_{11}$ (5.3.2)

what the simulator computes $|\psi\rangle = U_{\text{circuit}}|0\rangle^{\otimes n}, \quad p(x) = |\langle x|\psi\rangle|^2$ (5.4.1)

complex128 amplitudes: 16 GB at $n = 30$, 16 TB at $n = 40$ memory = $16 \cdot 2^n$ bytes (5.4.2)

the SX gate $\sqrt{X} = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}, \quad (\sqrt{X})^2 = X$ (5.5.1)

the Hadamard, in basis gates $H = e^{i\pi/4} R_z\left(\frac{\pi}{2}\right) \sqrt{X} R_z\left(\frac{\pi}{2}\right)$ (5.5.2)

CNOT from controlled-Z: Hadamards on the target $CX = (I \otimes H) CZ (I \otimes H)$ (5.5.4)

a SWAP from three CNOTs $\text{SWAP} = CX_{0 \rightarrow 1} CX_{1 \rightarrow 0} CX_{0 \rightarrow 1}$ (5.6.1)

estimated success probability from per-gate error rates ε_g $F \approx \prod_g (1 - \varepsilon_g) \approx \exp\left(-\sum_g \varepsilon_g\right)$ (5.6.3)

depth becomes time, and time becomes decoherence $t_{\text{circuit}} \approx d \cdot t_{\text{layer}}, \quad P_{\text{coherent}} \approx e^{-t_{\text{circuit}}/T_2}$ (5.7.1)

the GHZ state $|\text{GHZ}_n\rangle = \frac{|0\rangle^{\otimes n} + |1\rangle^{\otimes n}}{\sqrt{2}}$ (5.7.2)

equal up to a global phase $U \simeq V \Leftrightarrow \frac{|\text{Tr}(U^\dagger V)|}{2^n} = 1$ (5.8.1)

process fidelity: 1 for the same operation; the algorithm labs are marked with it $F_{\text{pro}}(U, V) = \frac{|\text{Tr}(U^\dagger V)|^2}{d^2}, \quad d = 2^n$ (5.8.2)